Where does sharing become limiting?

Three completed experiments on a 24-block, four-loop Transformer. A practical guide to the setup, code, and evidence.

Main finding
Independent per-loop LoRA beats a smaller shared adapter, but does not clearly beat the shared adapter with the same parameter count. Local gradient disagreement exists; a benefit from relaxing sharing is not yet established.

One question, three different measurements

Experiment	What we change	What it can tell us
1. Gradient directions	Nothing; measure backward contributions	Do repeated uses request aligned local changes?
2. Output sensitivity	Multiply one loop's attention or MLP output by 0.9 or 1.1	Is the model sensitive to that output scale?
3. LoRA controls	Train small added matrices; freeze the original model	Does per-loop adaptation beat shared adaptation?

What completed

All three experiments completed in about 6 hours 11 minutes using at most three H100 GPUs. All nine LoRA variants reached update 200, with final development and separate confirmation evaluations. The original model weights remained unchanged.

The run measured a final pretrained checkpoint only. It did not test early-training prediction, multiple training seeds, a new architecture from scratch, or equal total training FLOPs.

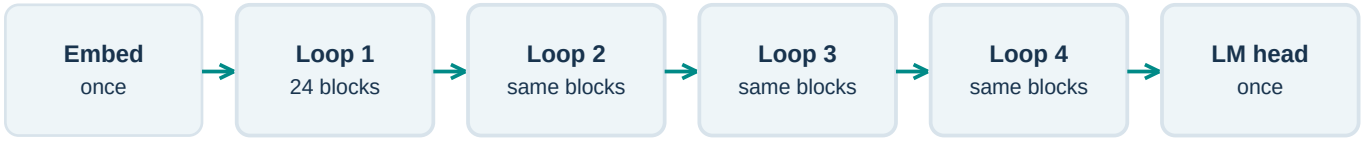
How to read this guide

Pages 2-6 explain the model and gradients; page 7 covers scaling; pages 8-10 cover LoRA and comparisons. Pages 11-12 map the code, reproduction steps, checks, and limits.

Code: research/looping_step1/ in RiddleHe/nanochat, target branch reasearch-looping. Numeric records accompany the code; datasets and checkpoints do not.

One set of blocks, four passes

Parameters are reused. Hidden states are recomputed at every use. This is not a 6-block model.



The output hidden state of loop 1 becomes the input to block 1 of loop 2. All four loops feed a single final language-model loss. There is no early-exit gate or loss attached to an intermediate loop.

Item	Recorded value
Physical / executed depth	24 shared blocks / 96 block executions
Width / attention heads	1,536 / 12 Q, K and V heads
Unique parameters	780,140,569 (about 780M)
Checkpoint / pretraining	Update 19,074 / 10,000,269,312 prediction tokens
Numerics	BF16 compute; FP32 master weights, loss and statistics; deterministic FA3 backward

Baseline details: full causal attention, rotary positions, Q/K normalization and ReLU-squared MLP. The smear embedding gate is retained. No value embeddings, x0 reinjection, AttnRes, per-loop LoRA, or intermediate-loop loss is in the original baseline.

What is a sequence?

A sequence is a list of token IDs. One stored record contains 2,049 IDs: x uses positions 1-2,048; the targets use positions 2-2,049. A micro-batch has two such examples, so its loss covers 4,096 target tokens.

> **A tiny next-token example**
> Stored: [The, cat, sat, down]
> Inputs x: [The, cat, sat] Targets y: [cat, sat, down]
> Real records follow the same shift, but contain 2,049 token IDs.

ClimbMix documents are joined with BOS separators and packed into fixed-length records. Attention is not reset at document boundaries. Exact document texts are deduplicated across the experimental splits; near duplicates are not audited.

What exactly is the gradient?

32 mini-batches x 2 sequences x 2,048 targets = 131,072 prediction tokens. No optimizer update.

A gradient describes how loss changes when weights change. Take one matrix W in layer 14, called four times. For each batch b, compute one mean token loss L_b after loop 4 and run backward through all four loops.

Separate contributions to the SAME parameter

```
G_1,b + G_2,b + G_3,b + G_4,b = dL_b / dW
```

G_2,b is the contribution made when W is used in loop 2. It is not a separate loop-2 loss, and it is not the gradient of the hidden state. Each G has the same shape as W.

How the hook computes a contribution

```
Z_r = X_r W^T
D_r = dL_b / dZ_r
G_r,b = D_r^T X_r
```

Object	Shape and meaning
X_r	4,096 x input width: inputs to this matrix use
D_r	4,096 x output width: backward signal from the final loss
G_r,b	output width x input width: contribution to W

The token average is already inside L_b and D_r. We do not divide G by 4,096 again. Matrix multiplication sums over token positions, including the loss's normalization. The hook matches the matrix-multiply dtype and converts contributions to FP32 for statistics.

From a matrix to a direction

Flatten G into a long vector g. This only rearranges its entries; it does not average them. For loops r and s in the same batch, calculate:

```
cosine_b(r,s) = dot(g_r,b, g_s,b)
                / (length(g_r,b) * length(g_s,b))
dot(u,v) = sum_i u_i*v_i; length(u) = sqrt(sum_i u_i*u_i)
```

+1 means aligned, 0 orthogonal, and -1 opposed. Moving along -g is the local descent direction; negating both vectors does not change their cosine. Adam also uses optimizer history and rescaling, so this is not an Adam-update map.

Correctness check
Every measured batch checked that the four contributions sum to autograd's shared-weight gradient. The largest recorded relative error was 4.263e-8. Code: interventions.py / VisitGradients; experiments.py / gradient.

Two averages, different questions

This distinction changes what the figure means. The two operations do not commute.

Current heatmap	A different, unmeasured statistic
Compute a cosine within each batch, then average the 32 numbers.	Average each loop's gradient vectors across batches, then compute their cosine.
Typical local alignment at the measured batch size.	Alignment of net gradients on the combined data.

```
Measured:    mean_b [ cos(g_2,b, g_3,b) ]
Not measured: cos( mean_b[g_2,b], mean_b[g_3,b] )
```

A two-number example: even the SIGN can change

These are invented vectors for teaching, not model measurements. Both batches have the same negative cosine:

Batch	Loop 2 gradient	Loop 3 gradient	Cosine
A	(4, 1)	(-1, -4)	-8/17 = -0.471
B	(-1, -4)	(4, 1)	-8/17 = -0.471
Mean vector	(1.5, -1.5)	(1.5, -1.5)	+1.000

Averaging the two cosines gives -0.471. Averaging the vectors first gives two identical directions, so their cosine is +1. Different batches can cancel each other's gradient components.

> **What the current colors do and do not say**
> Small- and large-gradient batches receive equal weight in the cosine mean. A red cell can indicate repeated local opposition without implying an opposed net gradient on all data. Changing batch size can change the result.

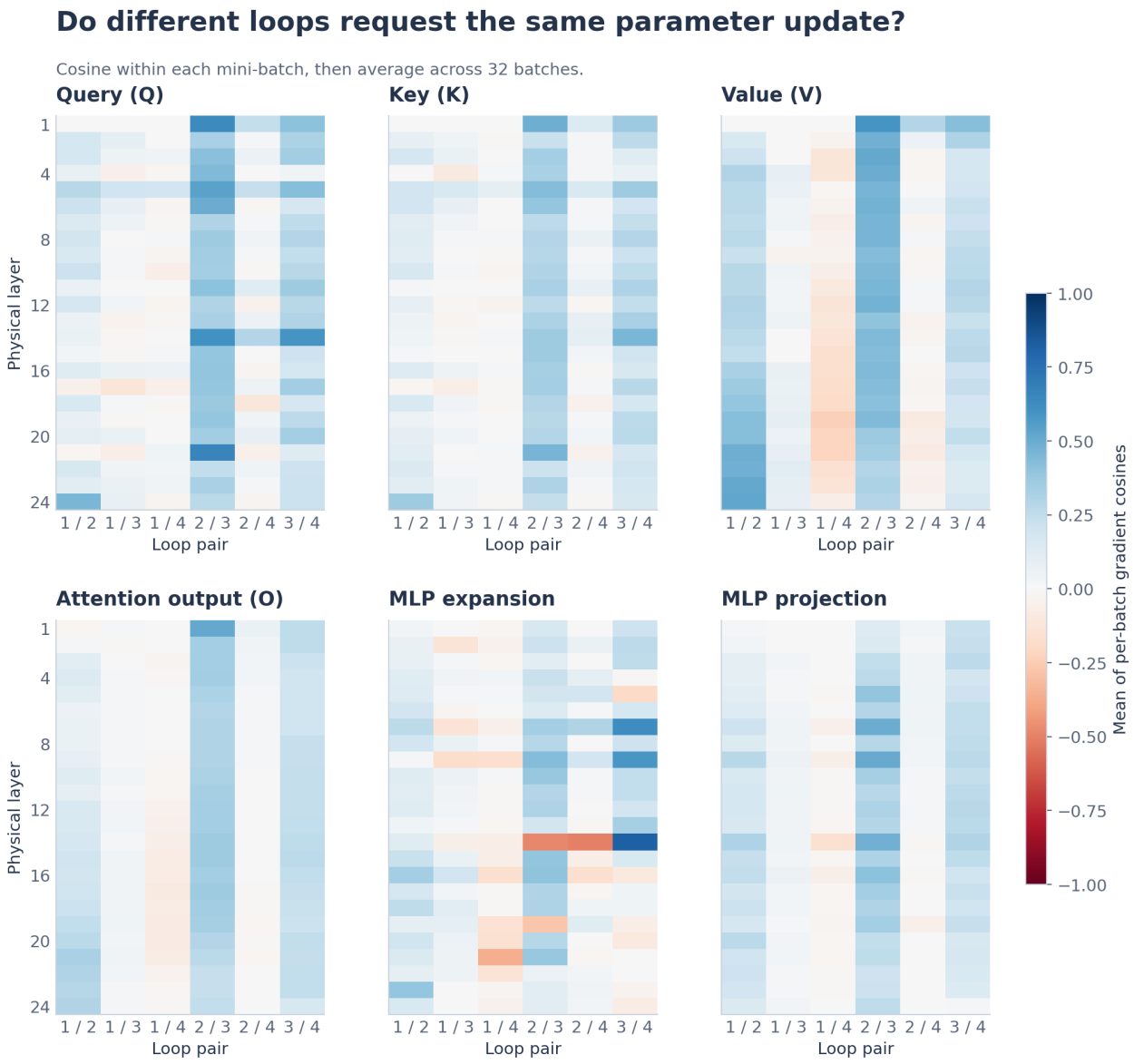
What we actually observed at layer 14

MLP expansion	Mean cosine	Negative batches	Range
Loops 2 / 3	-0.478904	30 / 32	-0.852 to +0.727
Loops 2 / 4	-0.502525	31 / 32	-0.789 to +0.029

These cells are not driven only by a few negative batches. But the mean-gradient cosine is still unknown. Stored cosines and norms cannot recover cross-batch vector dot products; a new gradient measurement is required, not model retraining.

Read the direction map correctly

Each colored cell summarizes 32 within-batch comparisons, not one global gradient comparison.



Red = opposing; blue = aligned. Not the cosine of the mean gradients.
Final checkpoint; 32 batches. Disagreement alone does not establish sharing harm.

Rows: physical layers 1-24. Columns: loop pairs, such as 2/3. Panels: Q, K, V, O, MLP expansion and MLP projection. Blue is aligned; red is opposed; white is near orthogonal. Color is not gradient magnitude.

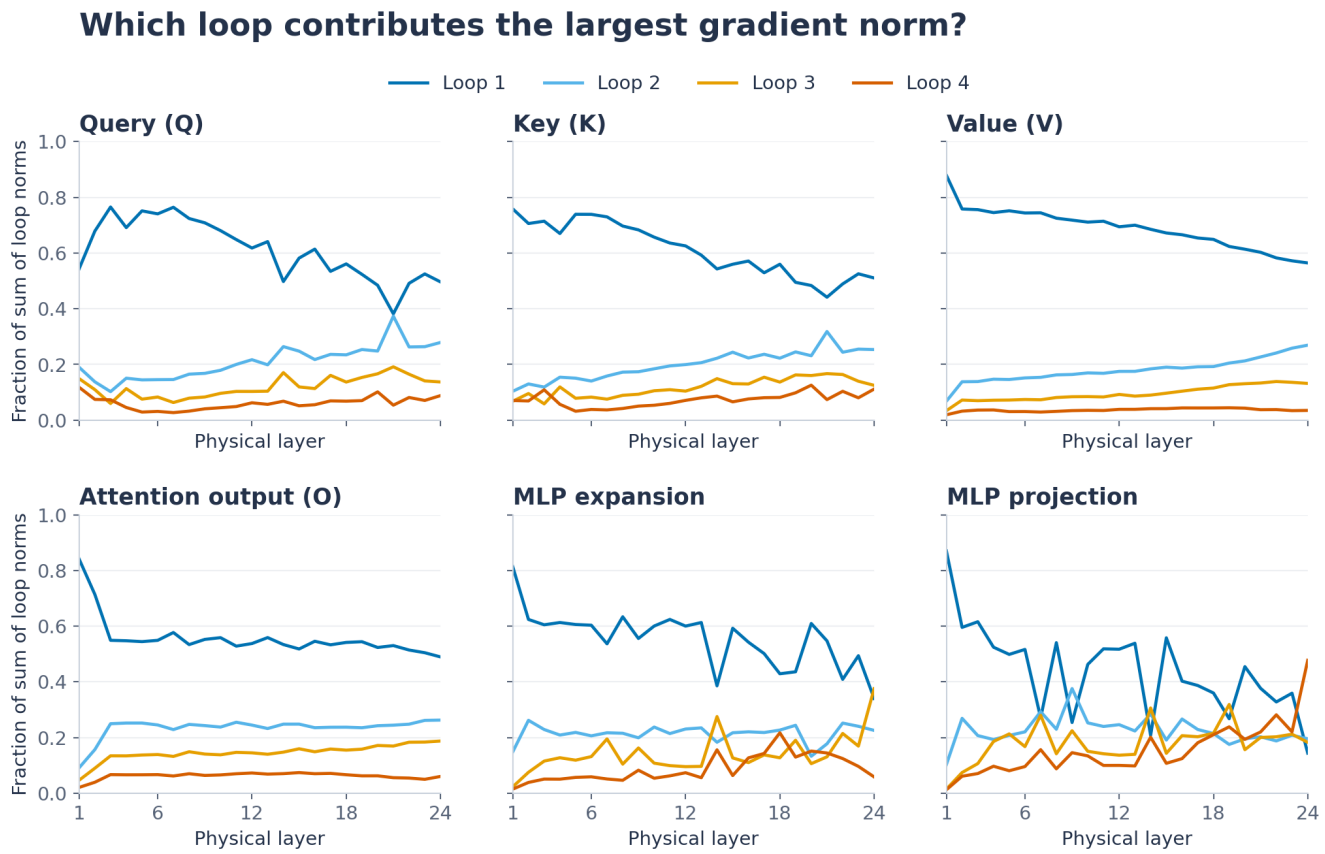
Example: the row-14, column-2/3 cell in MLP expansion compares that SAME matrix's loop-2 and loop-3 contributions. It does not compare layer 2 with layer 3, and it does not mean that the hidden state runs backward.

Interpretation boundary

A local negative cosine is a candidate diagnostic, not evidence that independent parameters will improve held-out quality. The map covers one final checkpoint and one fixed set of small batches.

Which loop has the larger signal?

A second statistic measures relative gradient length, not alignment or causal importance.



Average of per-batch norm shares · Four shares sum to one at each matrix
Norm magnitude is not a measure of causal importance.

```
share_r,b = length(g_r,b) / sum_s length(g_s,b)
plotted share_r = mean_b share_r,b
```

Horizontal axis: physical layer. Vertical axis: fraction of the sum of four gradient lengths. Dark blue is loop 1, light blue loop 2, yellow loop 3, and orange loop 4. Four shares sum to one for each matrix.

For V, the first-loop share averages about 69% across layers. Think of this as a louder local adjustment signal, not 69% of the model's useful reasoning or performance.

Three distinct questions
Direction: do two contributions point the same way?
Magnitude: how long is each contribution?
Causal value: does changing or removing a contribution improve the model?
Neither direction nor magnitude alone answers the third question.

Stored output: recorded/results/gradient/batches.jsonl retains every batch's cosine, norm, valid-norm flag, and gradient-sum error. results.json stores the aggregated map. Full gradient vectors were not retained.

Turn a module's output up or down

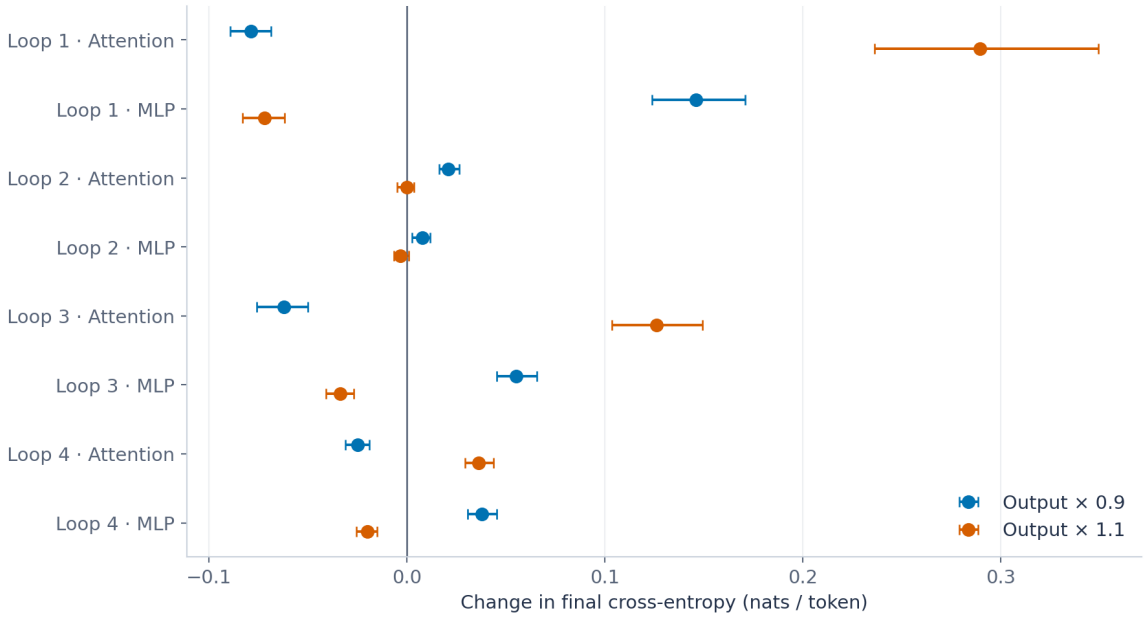
128 fixed sequences; 16 interventions; original weights frozen. This is a forward-only experiment.

```
Normal residual:  h_next = h + module(norm(h))
Intervention:     h_next = h + alpha * module(norm(h))
```

Choose one loop, one module class (attention or MLP), and alpha = 0.9 or 1.1. Apply it to that class in all 24 blocks of the chosen loop. This changes the residual branch output, not the whole hidden state and not the weight matrix.

What changes when a loop's module output is scaled?

Left of zero: lower loss. Right of zero: higher loss.



128 identical evaluation sequences · Each intervention affects one module class in all 24 layers of one loop
Whiskers: exploratory paired-sequence 95% bootstrap intervals; not adjusted for multiple comparisons.

Rows: loop/module combinations. Blue: output x0.9. Orange: output x1.1. Horizontal axis: change in final token cross-entropy; left of zero is better. Whiskers are descriptive paired-sequence 95% intervals.

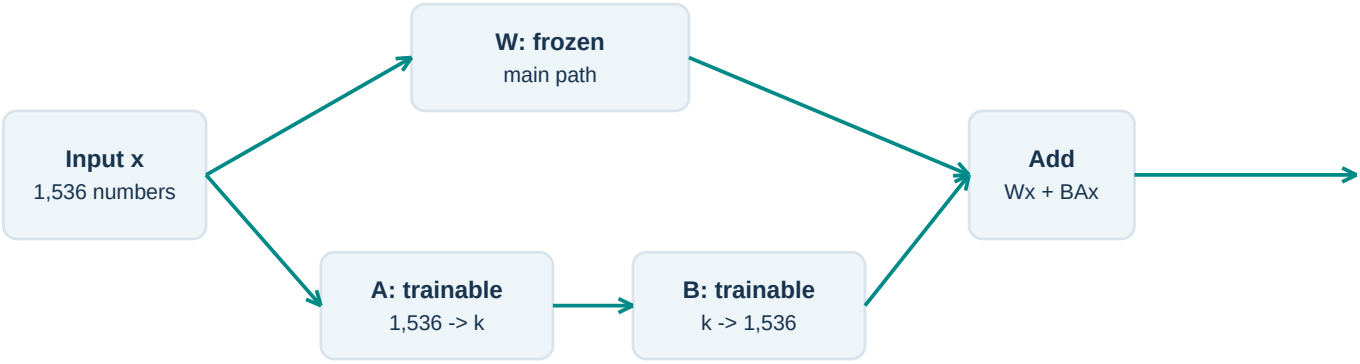
On this diagnostic selection, original CE is 2.726. Scaling loop-1 attention by 0.9 gives 2.647; separately, scaling loop-1 MLP by 1.1 gives 2.654. These gains cannot be added: the interventions were tested separately.

A competing explanation
Output calibration alone may help. A future unsharing method should therefore be compared with a simple output-scale control. These diagnostics are not a proof that shared parameters are the problem.

Code: interventions.py / ScaleOutputs; experiments.py / sensitivity. The alpha=1.0 sham was checked bitwise on the first two sequences, not all 128. The 16 comparisons are exploratory and not multiplicity-adjusted.

A small learned correction to W

LoRA leaves the original matrix frozen and adds a low-rank path. Only the small matrices learn.



Here k is the width of the small intermediate vector; the correction BA has rank at most k. For a 1,536 x 1,536 matrix and k=20, A and B add 20 x (1,536+1,536) = 61,440 parameters. They do not replace W: the original full-width path is still present.

For MLP expansion, the output is 6,144 instead of 1,536. Each adapter then has k x (1,536+6,144) parameters. Choosing k=8 again gives 61,440. Each experiment group targets two matrices in each of 24 blocks.

Variant	Q/K or V/O k	MLP k	Total added parameters
Shared small	20	8	2,949,120
Independent	20 per loop	8 per loop	11,796,480
Shared large	80	32	11,796,480

Shared variants use the same A and B in all four loops. Independent uses a separate pair per loop. Every B starts at zero, so all variants begin with exactly the original model's output. Independent A copies start identically to the small shared A.

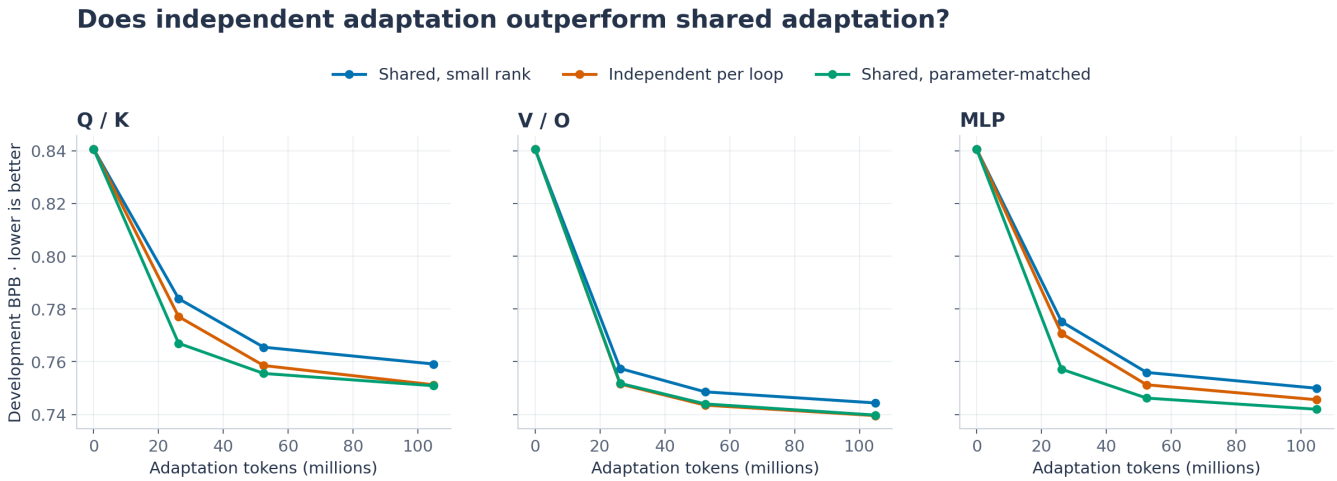
> **What 'relaxing sharing' means here**
> The large W remains shared and frozen. Only the small correction BA becomes loop-specific. This is not full untying of the Transformer weights. Backpropagation still passes through the frozen model to reach the adapters.

Code: interventions.py / LoopLoRA and attach_lora. Shared-large has four times the adapter-forward matrix FLOPs of independent. It does not have four times the total model FLOPs. Equal parameters is not equal total training compute.

Same data, three controlled variants

Three module groups x three sharing choices = nine independent adaptation runs.

Setting	Value
Batch and budget	2 x 2,048 tokens/micro-batch; 128 accumulated micro-batches/update; 200 updates = 104,857,600 tokens/run
Optimizer	AdamW; betas (0.9, 0.95), epsilon 1e-8; no weight decay, dropout or finite clipping
Learning rate	20-update warmup to 1e-4; cosine decay to 1e-5
Seed and evaluation	Seed 42; identical record order; dev/save at 0/50/100/200; separate confirmation at 200



Same frozen baseline and token order · One adaptation seed · 4,194,304 development tokens per evaluation
Parameter-matched shared adapters use more adapter FLOPs; these are not matched-total-compute comparisons.

Horizontal axis: extra training tokens in millions. Vertical axis: development BPB, lower is better. Blue is shared-small, orange independent, and green shared-large. Lines connect four measured evaluations; they are not continuous validation measurements.

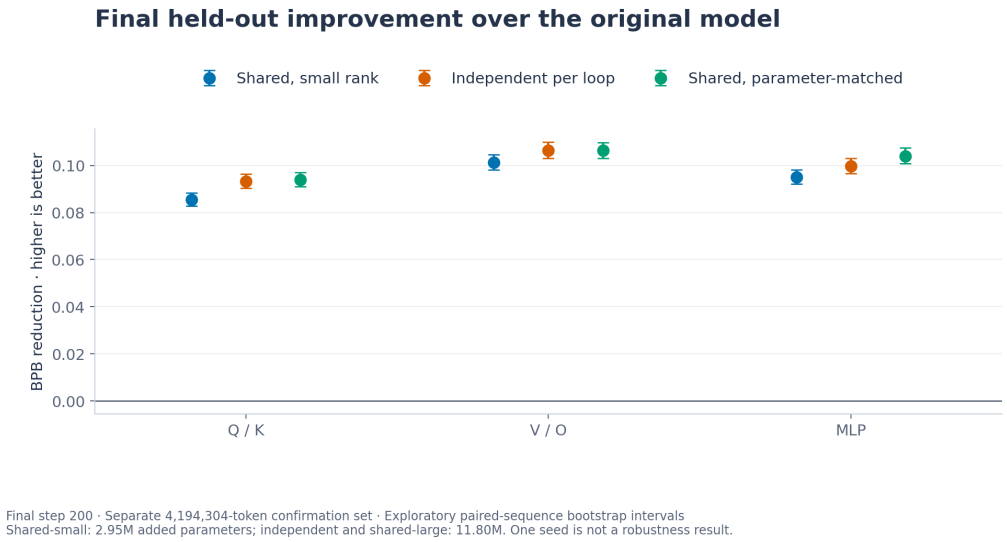
Terms in plain English

An update changes A and B after accumulating 128 small batches. Learning rate sets the size of an optimizer step. AdamW tracks moving averages of gradients; its weight-decay term is disabled here. Dropout would randomly mask adapter inputs during training; it is disabled. A seed makes initialization and data order repeatable, but one seed is not evidence of robustness.

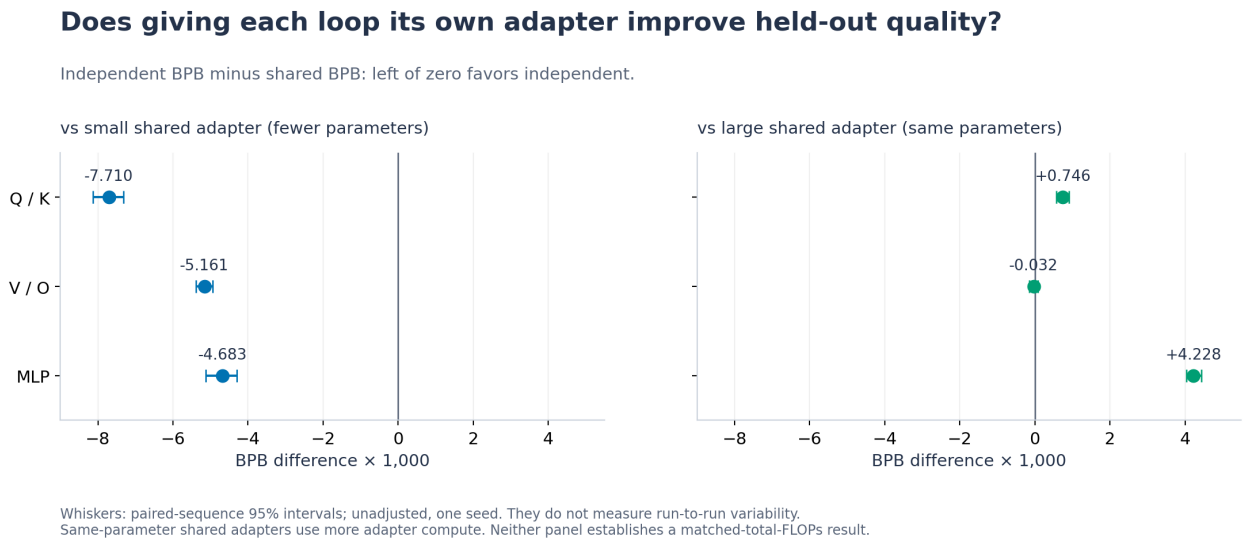
Development and confirmation each contain 2,048 sequences / 4,194,304 prediction tokens. The final update 200 is preselected for confirmation. We do not choose the best checkpoint by its confirmation score.

Compare against BOTH controls

All models are evaluated on the same confirmation sequences. Different packing or selections are not comparable.



Above: BPB reduction relative to the original model (higher is better). Colors match the learning curves. All variants improve, but each has extra training and parameters; this alone does not isolate a benefit from relaxed sharing.



Below: independent BPB minus the comparator BPB, multiplied by 1,000 for display. Left of zero favors independent. The left panel compares to shared-small; the right compares to parameter-matched shared-large.

Confirmation BPB	Shared small	Independent	Shared large
Q/K	0.766377	0.758668	0.757922
V/O	0.750682	0.745521	0.745553
MLP	0.756844	0.752161	0.747933

Against shared-large: Q/K is slightly worse; V/O is effectively tied (its interval crosses zero); MLP is worse. Original-model BPB on this same selection is 0.851886. No matched-total-FLOPs conclusion follows.

Find the implementation quickly

All paths below are relative to research/looping_step1/, unless marked as baseline files.

File	What to look for
common.py	SETTINGS; verified model/data loading; mean CE and byte-weighted BPB
interventions.py	VisitGradients, ScaleOutputs, LoopLoRA, attach_lora
experiments.py	gradient, sensitivity, lora, baseline_eval; per-batch cosine averaging
prepare.py	Document selection, packing, exact-text deduplication, freeze
launch.py	Three workers, GPU locks, queue, fail-without-auto-retry
tests.py / test_statistics.py	41 CPU algebra checks; real-model GPU preflight; aggregation regression tests
plot_results.py / analyze_results.py	Five plot families; validation and direct paired comparisons
recorded/results/	Per-batch statistics, training rows, per-sequence evaluations
docs/build_guide.py	This PDF; figures are in docs/figures/

CPU-only reproduction of the recorded analysis

```
# From repository root; Python + NumPy + Matplotlib
python research/looping_step1/test_statistics.py
python research/looping_step1/plot_results.py --recorded
python research/looping_step1/analyze_results.py
# ReportLab is required for this last command
python research/looping_step1/docs/build_guide.py
```

The first test uses only the Python standard library. Tiny-model tests additionally need PyTorch 2.10 and repository CPU dependencies: python research/looping_step1/tests.py cpu. None of these commands launches a GPU experiment.

Data selection is part of the setup

Diagnostics: ClimbMix train shard 300. Adaptation: ordered stream from shards 301-319, stopping when enough records are packed. Dev and confirmation: separate document selections from validation shard 6542. Exact-text duplicates are removed across these splits, but no near-duplicate audit is claimed.

The baseline lives in nanochat/model/gpt_loop.py and gpt_base.py. Packaging does not change either file. The run folder, original checkpoint and large packed arrays remain outside Git.

What is verified, and what is not

The packaged code preserves the numerical experiment. Portability changes are explicit, not hidden reruns.

A fresh GPU run needs the original frozen baseline

```
export LOOPING_BASELINE_DIR=/path/to/original-baseline
export LOOPING_STEP1_DIR=/path/to/NEW-step1-run
python research/looping_step1/prepare.py
python research/looping_step1/tests.py cpu
CUDA_VISIBLE_DEVICES=0 python research/looping_step1/tests.py gpu
python research/looping_step1/prepare.py --freeze
python research/looping_step1/launch.py
```

Use these GPU commands only with approved, free devices. The launcher is limited to GPUs 0/1/2. It starts gradient, sensitivity and the first LoRA job, then lets finished workers claim remaining jobs. It does not retry failed jobs. Never use the completed frozen run as the new output directory.

Metrics and uncertainty

CE averages loss in nats across all prediction tokens. BPB divides the loss on byte-bearing target tokens by ln(2) times the number of target bytes; zero-byte special tokens do not enter its numerator. Lower is better for both metrics.

The paired intervals use 2,000 resamples of the same sequence indices for both models, seed 42. BPB differences use a ratio of summed losses to summed bytes. Intervals are descriptive: they do not include seed variation, document clustering, or a correction for all exploratory comparisons.

Validation and provenance

Historical run: 41 CPU checks; nine real-size zero-LoRA equivalence checks; finite backward; exact adapter/optimizer resume; unchanged original weights. All 36 scheduled adapter checkpoints were checked on CPU after completion. Transient finite gradient spikes occurred and remain in the records; no configuration was changed to hide them.

Packaging revision: CPU tests and recorded-data analysis were rerun. H100 tests were not rerun because server access was unavailable. Runtime paths are now environment-configurable and the launcher uses the active Python interpreter. No new model training was performed.

> **The research decision**
> The next hypothesis is local: do high-conflict positions benefit more from selective unsharing than low-conflict positions, after controlling for output scaling? First confirm the gradient statistic at larger batch scales and with cosine of mean gradients. These are proposed tests, not completed findings.

Baseline commit: ad896a41c7e0426d7c4d349d3e4f64f3c9ebe421
Original experiment-manifest SHA256: f0d4708bff22bf428f865ac78e0b5d406e27a9e14124535904c6cf6b0f520084

Repository package and full setup | recorded/provenance.json contains the original source/data/checkpoint identifiers.